EXPERIMENT 28: PROLOG PROGRAM TO IMPLEMENT BEST FIRST SEARCH ALGORITHM

Aim

To write and execute a Prolog program to implement the Best First Search algorithm.

Objective

- To understand heuristic search.
- To represent a problem using a graph.
- To assign heuristic values to nodes.
- To select the most promising node based on its heuristic value.

Algorithm

1. Define the graph using edges.
2. Assign a heuristic value to each node.
3. Start from the initial node.
4. Place the initial node in the OPEN list.
5. Select the node having the lowest heuristic value.
6. Check whether the selected node is the goal.
7. If it is the goal, return the path.
8. Otherwise, expand the node and add its successors to the OPEN list.
9. Repeat the process until the goal is reached.

Flowchart

START

|

v

Initialize OPEN List

|

v

Select Node with

Lowest Heuristic

|

v

Is it Goal?

/ \

Yes No

| |

v v

Return Expand Node

Path

|

v

Add Successors

|

v

Update OPEN List

|

+------> Select Best Node

|

v

STOP

Code

```
% Best First Search


edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(c, g).

edge(e, h).
```

```
edge(f, h).

heuristic(a, 7).
heuristic(b, 6).
heuristic(c, 4).
heuristic(d, 7).
heuristic(e, 2).
heuristic(f, 3).
heuristic(g, 5).
heuristic(h, 0).

best_first(Start, Goal, Path) :-
    search([[Start]], Goal, ReversePath),
    reverse(ReversePath, Path).

search([[Goal|Rest]|_], Goal, [Goal|Rest]).

search([[Node|Rest]|Paths], Goal, Path) :-
    findall(
        [Next,Node|Rest],
        (edge(Node, Next),
         \+ member(Next, [Node|Rest])),
        NewPaths
    ),
    append(Paths, NewPaths, AllPaths),
    sort_paths(AllPaths, SortedPaths),
    search(SortedPaths, Goal, Path).

sort_paths(Paths, Sorted) :-
```

```
    map_list_to_pairs(path_value, Paths, Pairs),
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, Sorted).

path_value([Node|_], Value) :-
    heuristic(Node, Value).
```

Query

```
?- best_first(a, h, Path).
```

Output

```
Path = [a,c,f,h].
```

Result

The Best First Search algorithm was successfully implemented in Prolog using heuristic values.

Conclusion

Thus, Best First Search was successfully implemented using heuristic-based node selection.

```
5 ?- make.
% c:/users/varshini m/onedrive/
true.

6 ?- best_first(a, h, Path).
Path = [a, c, f, h]
```

Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPS

```
heuristic(f, 3).
heuristic(g, 5).
heuristic(h, 0).

best_first(Start, Goal, Path) :-
    search([[Start]], Goal, ReversePath),
    reverse(ReversePath, Path).

search([[Goal|Rest]|_], Goal, [Goal|Rest]).

search([[Node|Rest]|Paths], Goal, Path) :-
    findall(
        [Next,Node|Rest],
        (edge(Node, Next),
         \+ member(Next, [Node|Rest])),
        NewPaths
    ),
    append(Paths, NewPaths, AllPaths),
    sort_paths(AllPaths, SortedPaths),
    search(SortedPaths, Goal, Path).

sort_paths(Paths, Sorted) :-
    map_list_to_pairs(path_value, Paths, Pairs),
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, Sorted).

path_value([Node|_], Value) :-
    heuristic(Node, Value).
```